



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 11 - Part 1

Virtual Functions





THIS WEEK'S AGENDA

- 01. Virtual Functions
- 02. Virtual Destructors
- 03. Pure Virtual Functions
- 04. Abstract Classes





VIRTUAL FUNCTIONS





VIRTUAL FUNCTIONS

- In order to call the functions from the **actual object type**, rather than the **reference type**, we declare each such function as **virtual** in the base class.

```
virtual void display() {  
    std::cout << "Creating a Shape.." << std::endl;  
}
```





VIRTUAL FUNCTIONS

- The **virtual** reserved keyword must be used in the **base class**.
- All functions with the same name and parameter variable types in the derived classes are then automatically virtual.
- However, it is considered good practice to supply the **virtual** reserved keyword for the **derived class** functions as well.
- The keyword **override** is also used *after* the declaration of the virtual function.





VIRTUAL FUNCTIONS – ADVANTAGES

- Virtual functions give programs a lot of **flexibility**.
- The loop we ran on our Shape objects describes the general mechanism: “Display a shape object with it’s area”.
- Each object knows on its own how to carry out the specific tasks: “Display the question” and “Get the area of the shape”.
- Using virtual functions makes programs easily **extensible**.
- One or more new classes may be added as needed such as **Circle** or **Triangle**.
- All that would be needed is to override the virtual functions in the new classes with their correct implementations.





VIRTUAL DESTRUCTORS





VIRTUAL DESTRUCTORS

- The destructor of the base class should always be virtual.
- Suppose you use delete with a base class pointer that is pointing to an object of a derived class in order to destroy the derived class object.
- If the base class destructor is not virtual then delete, like a normal member function, calls the destructor for the base class and not the destructor for the derived class.
- This will cause only the base part of the object to be destroyed.





PURE VIRTUAL FUNCTIONS





PURE VIRTUAL FUNCTIONS

- A **pure virtual function** does not have an implementation (*but could there be exceptions?*)
- A **pure virtual function** is abstract, i.e. it exists only in name.
- From a syntactical point of view, a **pure virtual function** is declared by putting the keywords “= 0” after the function declaration. For example:

```
class Shape {  
    public:  
        // Pure virtual function  
        virtual void show() = 0;  
};
```





PURE VIRTUAL FUNCTIONS

- The **pure virtual function** is declared as shown below.
- The equal to sign here has nothing to do with assignment; the value 0 is not being assigned to anything.
- The “= 0” syntax is just how we tell the compiler that a virtual function will be pure.

```
class Shape {  
    public:  
        // Pure virtual function  
        virtual void show() = 0;  
};
```





ABSTRACT CLASSES





ABSTRACT CLASSES

- A class containing a pure virtual function is an **abstract class**.
- An abstract class is not instantiated, i.e. objects cannot be created out of an abstract class.
- An abstract class serves more like an interface or a template for derived classes.
- A derived class inherits all functions of the base class, including the pure virtual functions as well.
- If a derived class implements pure virtual functions by overriding them, then objects of the derived class can be instantiated. Otherwise, the derived class is also **abstract**.





SUMMARY





SUMMARY

- **Virtual functions** are a useful mechanism in C++ to implement polymorphism.
- Virtual functions begin with the keyword **virtual**.
- A derived class is supposed to override a virtual function inherited from the base class to exhibit polymorphic behavior.
- Pure virtual functions do not have any implementation.
- A class containing a pure virtual function is an **abstract class**.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 11





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

